

Intro to Universal Composability

Scott Griffy
University of Edinburgh

September 3rd, 2026

Intro and non-UC modeling

Goal of security modeling

Ensure protocol satisfies correctness, security (privacy/integrity)

Intro and non-UC modeling

Goal of security modeling

Ensure protocol satisfies correctness, security (privacy/integrity)

- ▶ Should allow for realistic attacks
 - ▶ Adversary can open multiple sessions at the same time
 - ▶ Capture computation, delays, randomness, adaptive corruption

Intro and non-UC modeling

Goal of security modeling

Ensure protocol satisfies correctness, security (privacy/integrity)

- ▶ Should allow for realistic attacks
 - ▶ Adversary can open multiple sessions at the same time
 - ▶ Capture computation, delays, randomness, adaptive corruption
- ▶ Dont allow unrealistic attacks
 - ▶ Adversary cannot see computation of honest parties, cannot choose keys for honest parties

Intro and non-UC modeling

Goal of security modeling

Ensure protocol satisfies correctness, security (privacy/integrity)

- ▶ Should allow for realistic attacks
 - ▶ Adversary can open multiple sessions at the same time
 - ▶ Capture computation, delays, randomness, adaptive corruption
- ▶ Dont allow unrealistic attacks
 - ▶ Adversary cannot see computation of honest parties, cannot choose keys for honest parties
- ▶ Be understandable

Intro and non-UC modeling

Baseline - Property-based modeling (not necessarily UC)

A PPT adversary plays a game we believe captures reality.

Example: CPA security:

Adversary $\mathcal{A}$

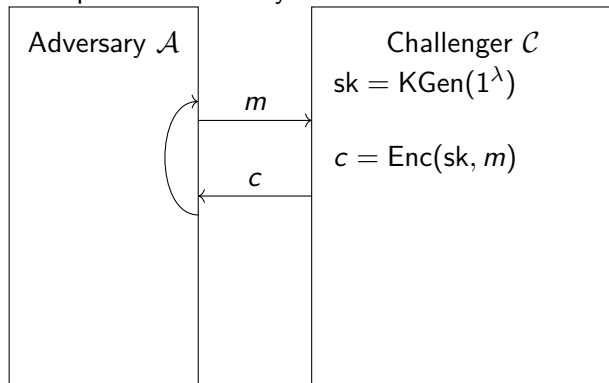
Challenger $\mathcal{C}$

Intro and non-UC modeling

Baseline - Property-based modeling (not necessarily UC)

A PPT adversary plays a game we believe captures reality.

Example: CPA security:

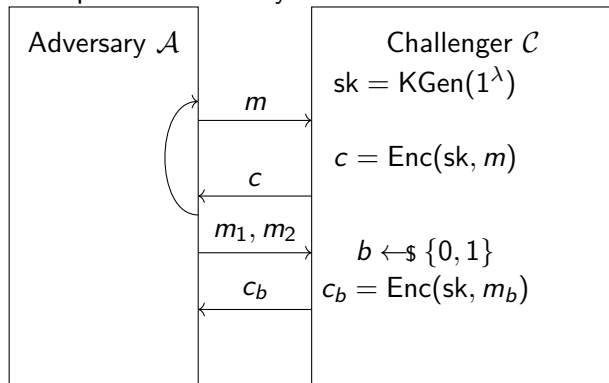


Intro and non-UC modeling

Baseline - Property-based modeling (not necessarily UC)

A PPT adversary plays a game we believe captures reality.

Example: CPA security:

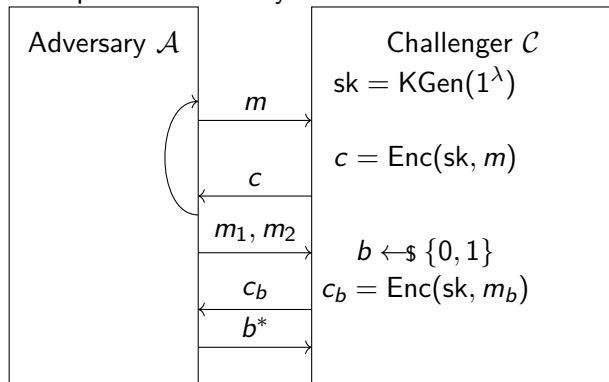


Intro and non-UC modeling

Baseline - Property-based modeling (not necessarily UC)

A PPT adversary plays a game we believe captures reality.

Example: CPA security:

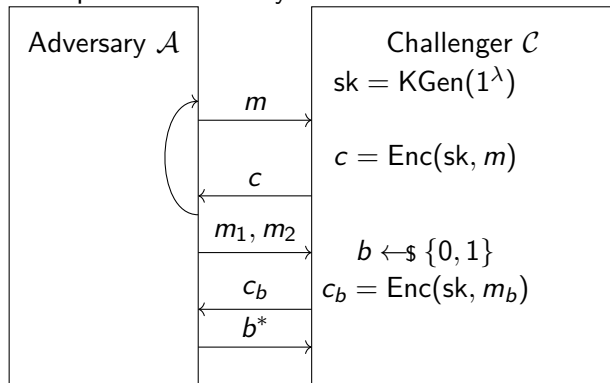


Intro and non-UC modeling

Baseline - Property-based modeling (not necessarily UC)

A PPT adversary plays a game we believe captures reality.

Example: CPA security:



Manually “composable”: can include in another protocol and use a reduction to play the game.

Problems with previous work

Why not property-based definitions?

Composability [Can00, MR92]

Non-malleability [DDN00]

Concurrency [DNS98]

Problems with previous work

Problems: composability

Key exchange and encryption

Needham-Schroeder:

A

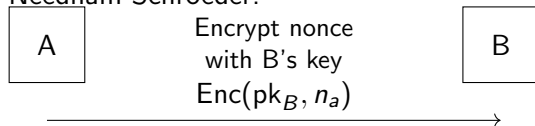
B

Problems with previous work

Problems: composability

Key exchange and encryption

Needham-Schroeder:

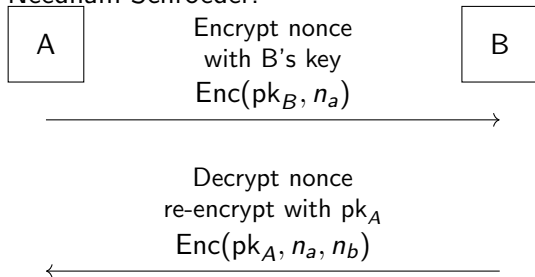


Problems with previous work

Problems: composability

Key exchange and encryption

Needham-Schroeder:

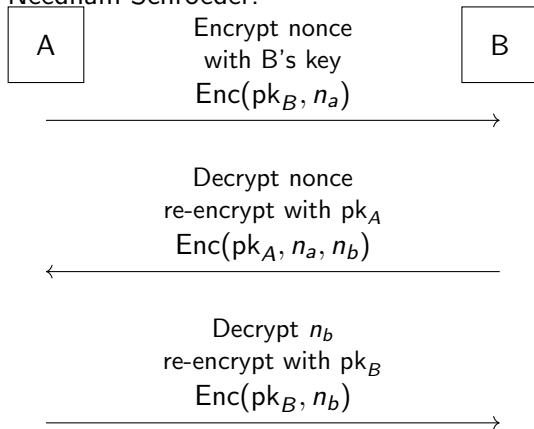


Problems with previous work

Problems: composability

Key exchange and encryption

Needham-Schroeder:

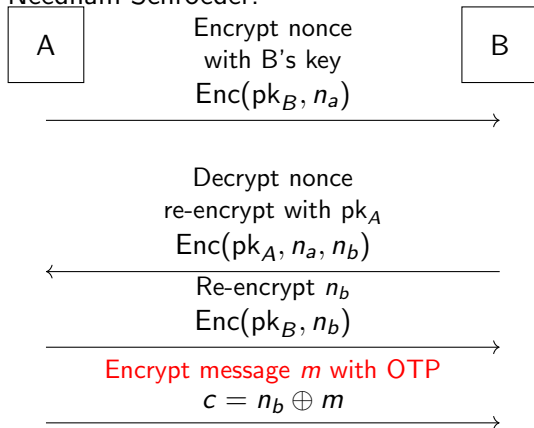


Problems with previous work

Problems: composability

Key exchange and encryption

Needham-Schroeder:

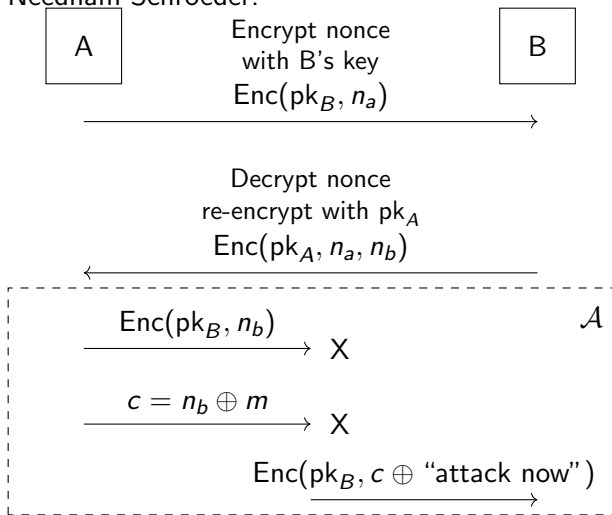


Problems with previous work

Problems: composability

Key exchange and encryption

Needham-Schroeder:



Problems with previous work

Composability

Could define “Primitive X with Y” but would have to redefine and re-prove each time (e.g. “Key exchange with encryption”, “Authentication and encryption” ... etc).

Problems with previous work

Problems: non-malleability

Alice encrypts “Alice has solved $P=NP$, here is the solution ...”

Non-malleability [DDN00]



Alice



Bob



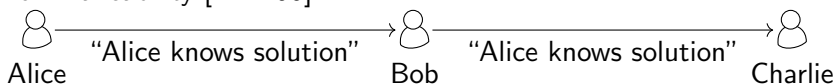
Charlie

Problems with previous work

Problems: non-malleability

Alice encrypts "Alice has solved $P=NP$, here is the solution ..."

Non-malleability [DDN00]

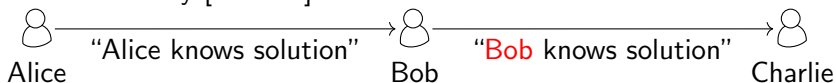


Problems with previous work

Problems: non-malleability

Alice encrypts "Alice has solved P=NP, here is the solution ..."

Non-malleability [DDN00]



Bob doesn't know the solution or secret key...

Stream cipher: $c' = c \oplus \text{"Alice"} \oplus \text{"Bob"}$

Problems with previous work

Problem: Concurrency

When multiple sessions are opened concurrently (with related state or requiring rewinding)



Problems with previous work

Problem: Concurrency

A trusts E, but B does not trust E



Problems with previous work

Problem: Concurrency

A trusts E, but B does not trust E



$\text{Enc}(\text{pk}_E, n_a)$



Problems with previous work

Problem: Concurrency

A trusts E, but B does not trust E



$\text{Enc}(\text{pk}_E, n_a)$



$\text{Enc}(\text{pk}_B, n'_a)$



Problems with previous work

Problem: Concurrency

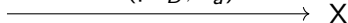
A trusts E, but B does not trust E



$\text{Enc}(\text{pk}_E, n_a)$



$\text{Enc}(\text{pk}_B, n'_a)$



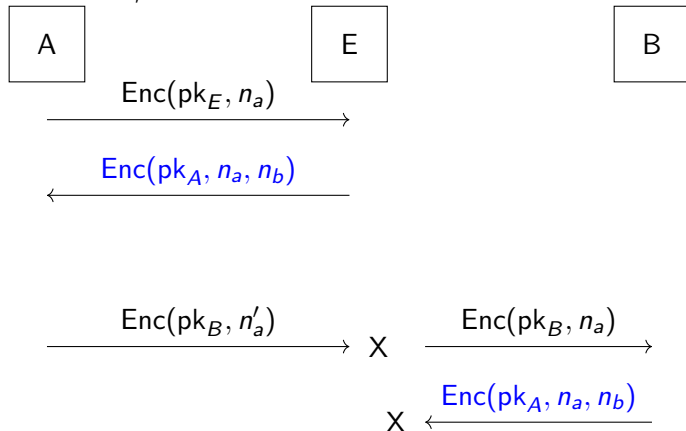
$\text{Enc}(\text{pk}_B, n_a)$



Problems with previous work

Problem: Concurrency

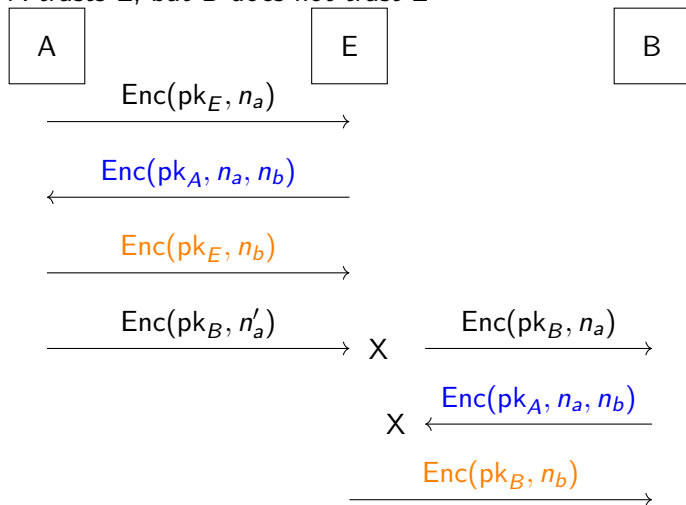
A trusts E, but B does not trust E



Problems with previous work

Problem: Concurrency

A trusts E, but B does not trust E



UC basic model

Functionality

Capture desired functionality for a system as an “ideal functionality” (a trusted third party).

UC basic model

Functionality

Capture desired functionality for a system as an “ideal functionality” (a trusted third party).

A real-world protocol, π , UC-realizes a functionality if any *environment* cannot distinguish it from the ideal functionality.

UC basic model

Functionality

Capture desired functionality for a system as an “ideal functionality” (a trusted third party).

A real-world protocol, π , UC-realizes a functionality if any *environment* cannot distinguish it from the ideal functionality.

Environment acts as an *interactive distinguisher* between the real and ideal worlds.

UC basic model

Functionality

Capture desired functionality for a system as an “ideal functionality” (a trusted third party).

A real-world protocol, π , UC-realizes a functionality if any *environment* cannot distinguish it from the ideal functionality.

Environment acts as an *interactive distinguisher* between the real and ideal worlds.

Real world: any PPT adversary interacts with the protocol

UC basic model

Functionality

Capture desired functionality for a system as an “ideal functionality” (a trusted third party).

A real-world protocol, π , UC-realizes a functionality if any *environment* cannot distinguish it from the ideal functionality.

Environment acts as an *interactive distinguisher* between the real and ideal worlds.

Real world: any PPT adversary interacts with the protocol

Ideal world: we simulate the adversary given leakage from the functionality (e.g. length of messages for encryption)

UC basic model

Functionality

Capture desired functionality for a system as an “ideal functionality” (a trusted third party).

A real-world protocol, π , UC-realizes a functionality if any *environment* cannot distinguish it from the ideal functionality.

Environment acts as an *interactive distinguisher* between the real and ideal worlds.

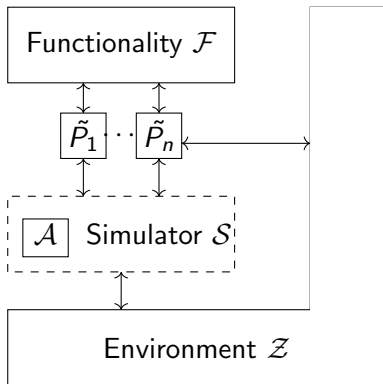
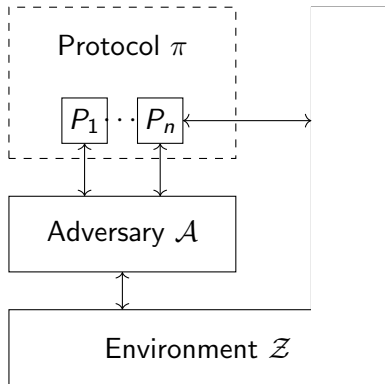
Real world: any PPT adversary interacts with the protocol

Ideal world: we simulate the adversary given leakage from the functionality (e.g. length of messages for encryption)

Introducing the environment and simulator allows for easy composition.

UC basic model

Diagram of UC security



Interactive turing machines

The setting

Model the real and ideal worlds as a system of Interactive Turing Machines (ITMs).

Interactive turing machines

The setting

Model the real and ideal worlds as a system of Interactive Turing Machines (ITMs).

ITMs include protocol parties $(P_1, \dots, P_n)$ and additional ITMs: Environment, Adversary, Simulator

Interactive turing machines

The setting

Model the real and ideal worlds as a system of Interactive Turing Machines (ITMs).

ITMs include protocol parties ($P_1, \dots, P_n$) and additional ITMs: Environment, Adversary, Simulator

Real world: Environment can talk to parties (input and output tapes) and adversary. Adversary can read and schedule protocol messages, corrupt parties, and respond to the environment.

Interactive turing machines

The setting

Model the real and ideal worlds as a system of Interactive Turing Machines (ITMs).

ITMs include protocol parties ($P_1, \dots, P_n$) and additional ITMs: Environment, Adversary, Simulator

Real world: Environment can talk to parties (input and output tapes) and adversary. Adversary can read and schedule protocol messages, corrupt parties, and respond to the environment.

Ideal world: Adversary is replaced by a simulator and parties are replaced by an ideal functionality. Simulator can schedule protocol messages, corrupt parties, access ideal leakage, and respond to the environment.

Interactive turing machines

The setting

Model the real and ideal worlds as a system of Interactive Turing Machines (ITMs).

ITMs include protocol parties ($P_1, \dots, P_n$) and additional ITMs: Environment, Adversary, Simulator

Real world: Environment can talk to parties (input and output tapes) and adversary. Adversary can read and schedule protocol messages, corrupt parties, and respond to the environment.

Ideal world: Adversary is replaced by a simulator and parties are replaced by an ideal functionality. Simulator can schedule protocol messages, corrupt parties, access ideal leakage, and respond to the environment.

Protocol “UC-secure” if environment can’t distinguish between the real and ideal world.

Interactive turing machines

Interactive Turing Machines (ITM)

ITMs can model any number of real-world physical systems
(computers on a network, processes on a computer, ...)

Interactive turing machines

Interactive Turing Machines (ITM)

ITMs can model any number of real-world physical systems
(computers on a network, processes on a computer, ...)

- ▶ input tape (honest inputs “Send” m to P_i)

Interactive turing machines

Interactive Turing Machines (ITM)

ITMs can model any number of real-world physical systems (computers on a network, processes on a computer, ...)

- ▶ input tape (honest inputs “Send” m to P_i)
- ▶ party I/O (interaction with parties (or functionality), scheduled by adversary)

Interactive turing machines

Interactive Turing Machines (ITM)

ITMs can model any number of real-world physical systems (computers on a network, processes on a computer, ...)

- ▶ input tape (honest inputs “Send” m to P_i)
- ▶ party I/O (interaction with parties (or functionality), scheduled by adversary)
- ▶ subroutine output tape (to call subroutines)

Interactive turing machines

Interactive Turing Machines (ITM)

ITMs can model any number of real-world physical systems (computers on a network, processes on a computer, ...)

- ▶ input tape (honest inputs “Send” m to P_i)
- ▶ party I/O (interaction with parties (or functionality), scheduled by adversary)
- ▶ subroutine output tape (to call subroutines)
- ▶ backdoor tape (for the adversary to control corrupted parties)

Interactive turing machines

Interactive Turing Machines (ITM)

ITMs can model any number of real-world physical systems (computers on a network, processes on a computer, ...)

- ▶ input tape (honest inputs “Send” m to P_i)
- ▶ party I/O (interaction with parties (or functionality), scheduled by adversary)
- ▶ subroutine output tape (to call subroutines)
- ▶ backdoor tape (for the adversary to control corrupted parties)

Many ways to formulate (Ex: In the ideal world, if there are only a static number of parties, each party could simply be another set of tapes on the ideal functionality)

Interactive turing machines

ITM Operation

- ▶ Some ITM is activated

Interactive turing machines

ITM Operation

- ▶ Some ITM is activated
- ▶ ITM reads its input tape

Interactive turing machines

ITM Operation

- ▶ Some ITM is activated
- ▶ ITM reads its input tape
- ▶ ITM performs some operation

Interactive turing machines

ITM Operation

- ▶ Some ITM is activated
- ▶ ITM reads its input tape
- ▶ ITM performs some operation
- ▶ ITM writes to its outgoing message tape and halts

Interactive turing machines

ITM Operation

- ▶ Some ITM is activated
- ▶ ITM reads its input tape
- ▶ ITM performs some operation
- ▶ ITM writes to its outgoing message tape and halts
- ▶ Environment is activated (or the party if this ITM was the environment)

Oblivious transfer

Oblivious Transfer Functionality

[CLOS02]

Functionality $\mathcal{F}_{\text{OT}}$

(Dummy) Parties: Sender: $\tilde{T}$, Receiver: $\tilde{R}$

Both $\tilde{T}$ and $\tilde{R}$ immediately pass their inputs from the environment to the functionality

- ▶ Upon receiving (send, sid, x_0, x_1) from $\tilde{T}$, record x_1, x_2
- ▶ Upon receiving (receive, $sid, i \in \{0, 1\}$) from $\tilde{R}$, send x_i to $\tilde{R}$ and sid to $\mathcal{S}$ (or nothing if no previous records)

Oblivious transfer

Oblivious Transfer Construction

Protocol π_{OT}

Real Parties: Sender: T , Receiver: R

- ▶ When T receives (send, sid , x_0 , x_1) from the environment, it chooses a pseudorandom permutation, f , f^{-1} and sends (f, sid) to R .
- ▶ When R receives (receive, sid , i) from the environment, it reads (f, sid) off its input tape, samples r randomly and sends back (sid, y_0, y_1) where $y_i = f(r)$ and y_{1-i} is sampled randomly.
- ▶ The next time T is activated, it reads (sid, y_0, y_1) from its input tape and computes $z_1 = f^{-1}(y_1) \oplus x_1$, $z_2 = f^{-1}(y_2) \oplus x_2$ and sends (sid, z_1, z_2) to R .
- ▶ The next time R is activated, it reads its input and outputs $x_i = r \oplus z_i$.

Oblivious transfer

Oblivious Transfer Proof

Simulator runs $\mathcal{A}$ as a black-box.

If T is corrupted, choose y_0, y_1 randomly. Indistinguishable because f is a permutation.

If R is corrupted, proceed normally, receive x_i from $\mathcal{F}$ (it is leaked since R is corrupted) choose $z_i = x_i \oplus f^{-1}(y_i)$ and z_{1-i} randomly to send back to R . As long as $\mathcal{A}$ cannot invert y_{1-i} to test if z_{1-i} really equals $x_{1-i} \oplus f^{-1}(y_{1-i})$, the simulation works.

UC composition

Composition setting

Given:

- ▶ π UC-realizes $\mathcal{G}$ in the $\mathcal{F}$ -hybrid model

UC composition

Composition setting

Given:

- ▶ π UC-realizes $\mathcal{G}$ in the $\mathcal{F}$ -hybrid model
- ▶ For instance: π realizes secure communication given ideal authenticated channels.

UC composition

Composition setting

Given:

- ▶ π UC-realizes $\mathcal{G}$ in the $\mathcal{F}$ -hybrid model
 - ▶ For instance: π realizes secure communication given ideal authenticated channels.
- ▶ ρ UC-realizes $\mathcal{F}$.

UC composition

Composition setting

Given:

- ▶ π UC-realizes $\mathcal{G}$ in the $\mathcal{F}$ -hybrid model
 - ▶ For instance: π realizes secure communication given ideal authenticated channels.
- ▶ ρ UC-realizes $\mathcal{F}$.

Want to construct π^* in the real world (not a hybrid model)

UC composition

Composition setting

Given:

- ▶ π UC-realizes $\mathcal{G}$ in the $\mathcal{F}$ -hybrid model
 - ▶ For instance: π realizes secure communication given ideal authenticated channels.
- ▶ ρ UC-realizes $\mathcal{F}$.

Want to construct π^* in the real world (not a hybrid model)

Solution: add translators that converts calls to the ideal functionality into calls to the protocol, ρ .

UC composition

Composition setting

Given:

- ▶ π UC-realizes $\mathcal{G}$ in the $\mathcal{F}$ -hybrid model
 - ▶ For instance: π realizes secure communication given ideal authenticated channels.
- ▶ ρ UC-realizes $\mathcal{F}$.

Want to construct π^* in the real world (not a hybrid model)

Solution: add translators that converts calls to the ideal functionality into calls to the protocol, ρ . We will have one for each party (σ_i^ρ).

UC composition

Composition setting

Given:

- ▶ π UC-realizes $\mathcal{G}$ in the $\mathcal{F}$ -hybrid model
 - ▶ For instance: π realizes secure communication given ideal authenticated channels.
- ▶ ρ UC-realizes $\mathcal{F}$.

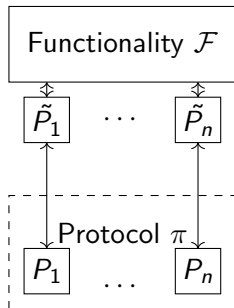
Want to construct π^* in the real world (not a hybrid model)

Solution: add translators that converts calls to the ideal functionality into calls to the protocol, ρ . We will have one for each party (σ_i^ρ).

Universal composition theorem: π^* (with the translators)
UC-emulates π (and thus, by transitivity, π^* also UC-realizes $\mathcal{G}$).

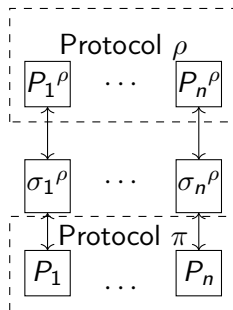
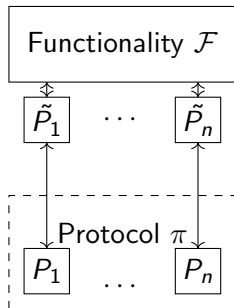
UC composition

New protocol



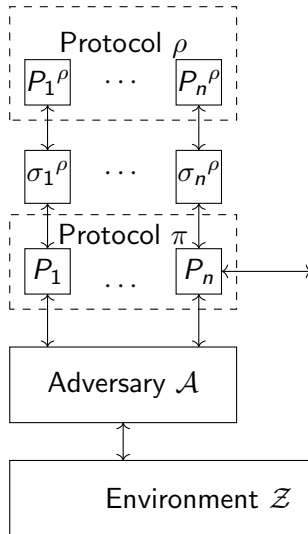
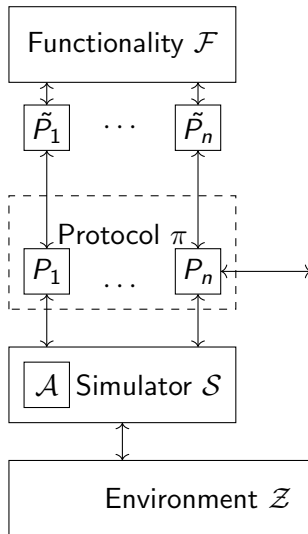
UC composition

New protocol



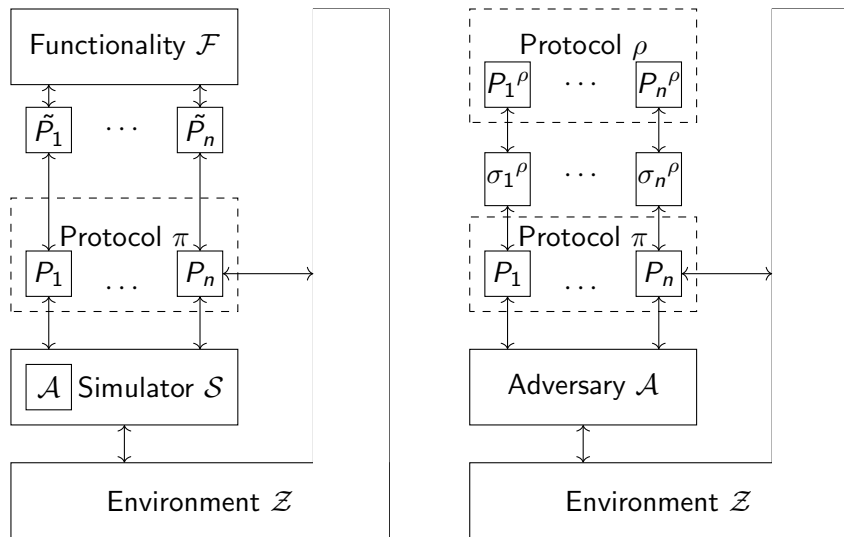
UC composition

Composition game



UC composition

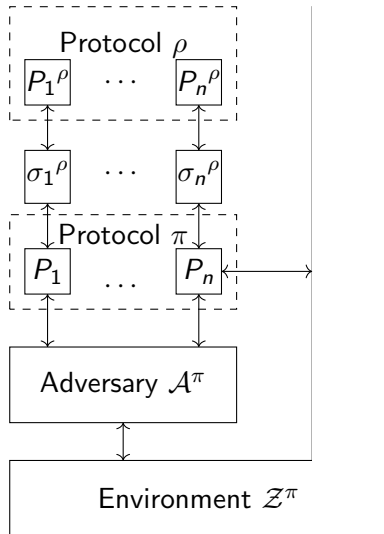
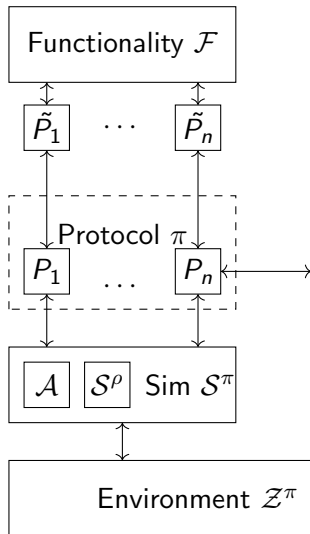
Composition game



Base our simulator on $\mathcal{S}^\rho$ (simulator for UC-security of ρ)

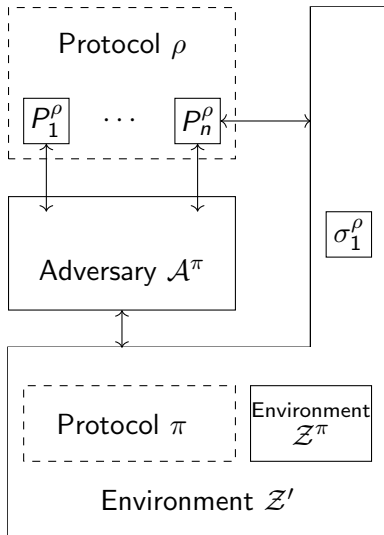
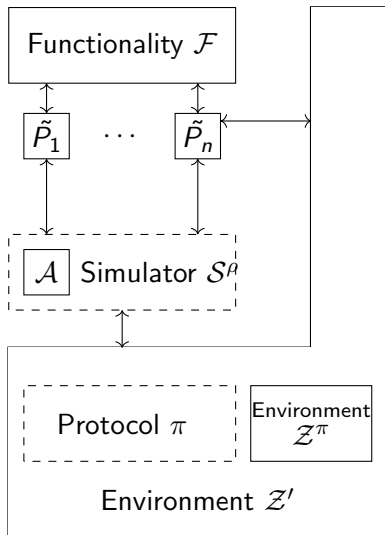
UC composition

Composition game



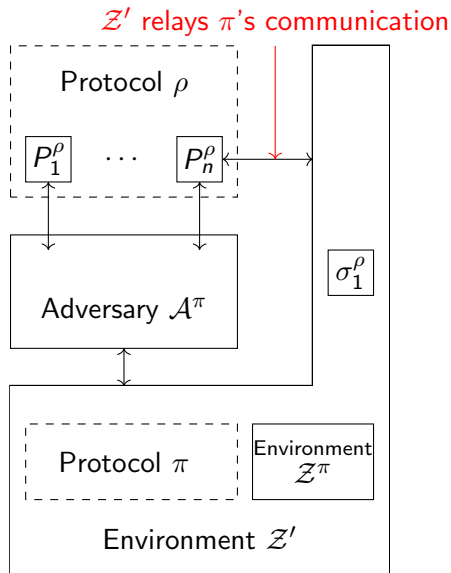
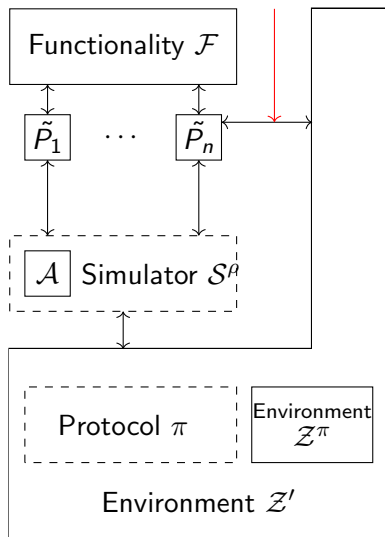
UC composition

Reducing to UC-security of ρ



UC composition

Reducing to UC-security of ρ



Interesting Details

UC-security

**Unbalanced Environment:
Trivial Realizations of functionalities:**

Interesting Details

UC-security

Unbalanced Environment: Could be impossible to UC-emulate a protocol if the environment doesn't give the simulator enough input (simulator must be polynomial time based on its input). This is an “unbalanced environment”.

Trivial Realizations of functionalities:

Interesting Details

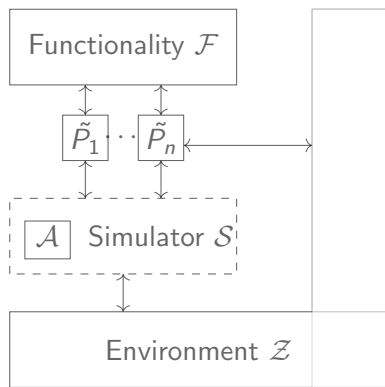
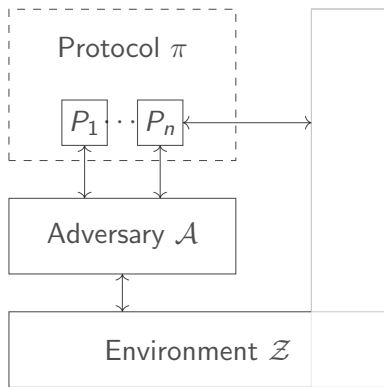
UC-security

Unbalanced Environment: Could be impossible to UC-emulate a protocol if the environment doesn't give the simulator enough input (simulator must be polynomial time based on its input). This is an “unbalanced environment”.

Trivial Realizations of functionalities: A protocol could be trivially easy to realize if the protocol and simulator simply hang (thus, preventing the environment from using anything more than the API in both cases).

Conclusion

Thanks you! Questions?





Ran Canetti.

Security and composition of multiparty cryptographic protocols.

Journal of Cryptology, 13(1):143–202, January 2000.



Ran Canetti, Yehuda Lindell, Rafail Ostrovsky, and Amit Sahai.

Universally composable two-party and multi-party secure computation.

In *34th ACM STOC*, pages 494–503. ACM Press, May 2002.



Danny Dolev, Cynthia Dwork, and Moni Naor.

Nonmalleable cryptography.

SIAM Journal on Computing, 30(2):391–437, 2000.



Cynthia Dwork, Moni Naor, and Amit Sahai.

Concurrent zero-knowledge.

In *30th ACM STOC*, pages 409–418. ACM Press, May 1998.



Silvio Micali and Phillip Rogaway.

Secure computation (abstract).

In Joan Feigenbaum, editor, *CRYPTO'91*, volume 576 of *LNCS*, pages 392–404. Springer, Berlin, Heidelberg, August 1992.